

**CSA0911 – JAVA PROGRAMMING**  
**COMMON COURSE ASSIGNMENT FORMAT**  
**SMART HOSPITAL PATIENT APPOINTMENT, PHARMACY INVENTORY**  
**AND ALERT NOTIFICATION SYSTEM**

**Attaluri Sri Vamsi (192372242)**

**P.V.S.Pradeep Reddy(192311364)**

**B.Hemanth Kumar(192324238)**

**A. Assignment Information**

| Particular                | Details                                                                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Department:               | Computer Science and Engineering (Artificial Intelligence)                                                                                                                                                                   |
| Programme:                | B.E.                                                                                                                                                                                                                         |
| Course Code & Course Name | CSA0911 - Programming in Java                                                                                                                                                                                                |
| Academic Year / Batch     | 2023-2027                                                                                                                                                                                                                    |
| Faculty Name              | Dr.R.Thalapathi Rajasekaran                                                                                                                                                                                                  |
| Assignment Title          | Hospital Management System using Java OOP, Collections, Exception Handling and Multithreading                                                                                                                                |
| Date of Issue             | 01-September -2026                                                                                                                                                                                                           |
| Date of Submission        | 02-September-2026                                                                                                                                                                                                            |
| Maximum Marks             | 100                                                                                                                                                                                                                          |
| Course Outcome(s) - CO    | CO1: Apply OOP principles in Java.<br>CO2: Implement Java Collections & Iterators.<br>CO3: Develop robust applications using custom exceptions.<br>CO4: Design concurrent multithreaded systems with thread synchronization. |

| Particular                    | Details                                                                                                                      |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Bloom's Taxonomy Level        | Level 3 (Apply), Level 4 (Analyze), Level 5 (Evaluate), Level 6 (Create)                                                     |
| SDG Mapping (SDG 1-17)        | SDG 3: Good Health and Well-being<br>SDG 9: Industry, Innovation, and Infrastructure<br>SDG 10: Reduced Inequalities         |
| Industry / Societal Relevance | Hospital appointment workflows, pharmacy safety, and concurrent race-condition prevention in critical health infrastructure. |

## 1. Problem Statement and Problem Formulation

The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System is a complete, menu-driven Core Java application designed to manage patient health records, doctor appointment scheduling, pharmacy medicines, inventory levels, and notification activities in an integrated, thread-safe manner.

The system addresses common hospital-management challenges such as registering patients and doctors, displaying records, booking and cancelling appointments, maintaining a fair FIFO waitlist when doctors are fully booked, issuing medicines atomically, restocking supplies, detecting low-stock medicines, generating reports, and dispatching asynchronous notifications.

The problem is formulated as an interconnected collection of domain entities (Person, Patient, Doctor, Appointment, Medicine, Inventory, Notification). Shared state information is protected using private attributes (Encapsulation), abstract base classes define unified contracts (Abstraction), specialized patient categories and medicine types leverage inheritance and method overriding (Polymorphism), Java Collections manage dynamic data with explicit Iterators, custom checked exceptions protect application flow, and multithreading with synchronization locks simulates concurrent bookings and asynchronous alert dispatching.

## 2. Objective and Expected Outcomes

- Develop a comprehensive menu-driven hospital-management application using Core Java.
- Apply classes, objects, encapsulation, data hiding, inheritance, and runtime polymorphism.
- Use ArrayList, HashMap, LinkedList, and Queue with Java generics for safe record management.

- Explicitly implement Iterator and ListIterator for collection traversal, safe removal, and in-place updates.
- Implement user-defined custom checked exceptions for invalid IDs, duplicate appointments, and out-of-stock conditions.
- Implement multithreading, thread priorities, synchronization, and race-condition prevention.
- Maintain doctor appointment capacity and automatic FIFO waitlist promotion upon appointment cancellation.
- Generate real-time patient visit, appointment status, revenue, and pharmacy inventory reports.
- Dispatch asynchronous appointment reminders and critical low-stock notifications.
- Provide a reliable, crash-resilient console user interface with robust input validation.

### 3. Requirements, Constraints and Assumptions

#### 3.1 Functional Requirements

- Register, display, search, and update patient records across General, Senior Citizen, and Insured tiers.
- Register and display doctors with specializations and available consultation slots.
- Book and cancel appointments with conflict detection for duplicate date/time slots.
- Automatically place patients onto a doctor's waitlist when consultation slots are occupied.
- Promote the next waitlisted patient to a booked appointment when an existing appointment is cancelled.
- Add, display, dispense, and restock Prescription (Rx) and Over-The-Counter (OTC) medicines.
- Detect medicines at or below the configured low-stock safety threshold ( $\leq 10$  units).
- Dispatch polymorphic appointment reminder and low-stock alert notifications in the background.
- Generate comprehensive summary reports detailing patient visits, physician metrics, and inventory value.
- Simulate concurrent multi-user booking contention using threads and thread priorities.

#### 3.2 Non-Functional Requirements

- The program must be easy to operate through an intuitive numbered console menu.
- Shared appointment schedules and inventory quantities must remain strictly consistent and thread-safe.
- Invalid operations and exceptions must be reported cleanly without terminating the application.
- The implementation must strictly adhere to standard Core Java without external third-party dependencies.

### 3.3 Constraints

- The application is console-based using standard Scanner input.
- Data is maintained in-memory during application runtime without external database persistence.
- Notification dispatching is simulated in the console through background worker threads.

### 3.4 Assumptions

- Patient IDs, Doctor IDs, and Medicine IDs are unique identifiers.
- Standard base consultation fee is \$500.00. Senior citizens receive a 40% waiver (\$300.00), and insured patients pay a co-pay (\$150.00).
- Critical inventory safety threshold is set at  $\leq 10$  units.

## 4. Application of Relevant Course Knowledge / Concepts

| Course Concept              | Implementation in Project                                                                                         | Academic Purpose                                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Classes and Objects         | Patient, Doctor, Appointment, Medicine, Inventory, Notification                                                   | Models real-world hospital domain entities into software components.          |
| Encapsulation / Data Hiding | Private fields with public validated constructors, getters, and setters                                           | Protects internal state and enforces validation constraints.                  |
| Inheritance                 | GeneralPatient, SeniorPatient, InsuredPatient extends Patient; PrescriptionMedicine, OTCMedicine extends Medicine | Establishes hierarchical relationships and promotes code reuse.               |
| Polymorphism                | Overridden getConsultationFee(), getDosageRule(), and send()                                                      | Enables runtime dynamic method dispatch based on instantiated object type.    |
| ArrayList                   | Doctor.appointments, HospitalManagementSystem.allAppointments                                                     | Stores ordered, indexed collections of appointment records.                   |
| HashMap                     | HospitalManagementSystem.patients, Inventory.medicineMap                                                          | Provides fast $O(1)$ key-based lookup for patient IDs and medicine stock.     |
| LinkedList / Queue          | Doctor.waitlist, HospitalManagementSystem.notificationQueue                                                       | Implements FIFO queuing for waitlisted patients and background notifications. |

| Course Concept     | Implementation in Project                                                                                                        | Academic Purpose                                                                    |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Iterator           | Traversing patients, doctors, medicines, and appointments; safe removal via <code>iterator.remove()</code>                       | Traverses collections safely without <code>ConcurrentModificationException</code> . |
| ListIterator       | Appointment and Patient updates via <code>hasPrevious()</code> , <code>previous()</code> , and <code>set()</code>                | Supports bidirectional list traversal and in-place element replacement.             |
| Generics           | <code>ArrayList&lt;Appointment&gt;</code> , <code>HashMap&lt;String, Patient&gt;</code> , <code>Queue&lt;Notification&gt;</code> | Ensures compile-time type safety and eliminates manual type casting.                |
| Exception Handling | <code>InvalidPatientIdException</code> , <code>DuplicateAppointmentException</code> , <code>OutOfStockException</code>           | Handles invalid operations and runtime conflicts gracefully.                        |
| Multithreading     | <code>BookingThread</code> , <code>NotificationDispatchThread</code> implementing <code>Runnable</code>                          | Simulates concurrent user booking requests and background processing.               |
| Synchronization    | synchronized methods and lock objects ( <code>appointmentLock</code> , <code>notificationLock</code> )                           | Protects shared mutable state against race conditions and corruptions.              |
| Thread Priorities  | <code>Thread.MIN_PRIORITY</code> (1), <code>Thread.NORM_PRIORITY</code> (5), <code>Thread.MAX_PRIORITY</code> (10)               | Demonstrates OS thread scheduling hints vs. deterministic locks.                    |

## 5. Design / Proposed Solution / Methodology

The proposed architecture follows a layered object-oriented software engineering design. Entity classes represent domain data, while `HospitalManagementSystem` acts as the service coordinator. `Inventory` manages pharmacy stock operations, and `NotificationDispatchThread` processes alerts asynchronously.

### 5.1 Major Modules

- **Patient Management Module:** Handles registration, search, display, and updates with polymorphic fee calculations.
- **Doctor Management Module:** Manages doctor profiles, specialization details, consultation slots, and FIFO waitlists.

- Appointment Module: Manages consultation bookings, conflict detection, cancellations, and automatic waitlist promotion.
- Pharmacy Inventory Module: Handles medicine additions, atomic stock dispensing, restocking, and low-stock detection.
- Notification Module: Asynchronously dispatches polymorphic appointment reminders and pharmacy warnings.
- Reporting Module: Computes real-time statistics on patient visits, doctor loads, revenue, and inventory valuation.
- Concurrency Module: Simulates multi-threaded slot contention using thread priorities and synchronization locks.
- Menu & Input Module: Provides a robust, crash-resilient 14-option user interface.

## 5.2 Data Flow

User input is accepted via Scanner helper methods in Main.java. Operations route to HospitalManagementSystem, which validates records and updates collections. Booking and dispensing events generate Notification objects pushed into notificationQueue. A background worker thread asynchronously polls and delivers notifications.

## 6. Algorithm / Pseudocode / Flowchart

### 6.1 Main System Algorithm

- Step 1: Start application and initialize HospitalManagementSystem with preloaded seed records.
- Step 2: Display numbered main menu options (1 to 14).
- Step 3: Read and validate user choice.
- Step 4: Execute selected operation within structured try-catch blocks:
  - If Booking: Validate patient/doctor -> Check slot -> If booked, throw DuplicateAppointmentException and enqueue waitlist; If free, create BOOKED appointment and queue reminder.
  - If Cancellation: Search appointment with Iterator -> Mark CANCELLED -> Poll doctor waitlist -> If waiting patient exists, promote to BOOKED on vacated slot.
  - If Dispense: Verify stock in Inventory -> Deduct atomically -> If stock < requested, throw OutOfStockException -> If remaining <= 10, trigger LowStockAlert.
  - If Concurrency Demo: Launch 3 threads with priorities 1, 5, 10 contending for the same slot -> Verify synchronization lock integrity.
- Step 5: Catch and display domain-specific custom exceptions gracefully.
- Step 6: Repeat menu loop until Option 14 (Exit) is selected.

### 6.2 Comprehensive Pseudocode

```
BEGIN MAIN_PROGRAM
```

```

Create HospitalManagementSystem instance
Load sample patients, doctors, and medicines
REPEAT
    Display Main Menu (Options 1 to 14)
    Read user choice
    TRY
        CASE choice OF
            1: Register Patient (General / Senior / Insured)
            2: Register Doctor
            3: Add Medicine (Prescription / OTC)
            4: Book Appointment (Synchronized slot booking)
            5: Cancel Appointment (Synchronized with automatic waitlist
promotion)
            6: Search Patient (Using Iterator)
            7: Update Patient (Using ListIterator)
            8: View Doctor Appointments
            9: View Waitlist
            10: View Inventory & Dispense Medicine
            11: Dispatch Notifications (Asynchronous background worker)
            12: Run Concurrent Booking Test (Thread Priorities: 1, 5, 10)
            13: Generate Comprehensive Hospital Report
            14: Exit Program
        END CASE
    CATCH InvalidPatientIdException e
        Display "[CUSTOM EXCEPTION] Invalid Patient ID: " + e.getMessage()
    CATCH DuplicateAppointmentException e
        Display "[CUSTOM EXCEPTION] Duplicate Appointment: " + e.getMessage()
        Prompt user to add patient to doctor's waitlist queue
    CATCH OutOfStockException e
        Display "[CUSTOM EXCEPTION] Out Of Stock: " + e.getMessage()
    CATCH Exception e
        Display "[ERROR] " + e.getMessage()
    END TRY
    UNTIL choice == 14
END MAIN_PROGRAM

```

### 6.3 Flowchart Description

Start -> Initialize System -> Load Seed Records -> Display Menu -> Read Choice -> Validate Input -> Route to Service Module -> Execute Synchronized Logic -> Handle Domain Exceptions -> Update Collection State -> Enqueue Alerts -> Display Output -> Exit chosen? If No, repeat Menu; If Yes, terminate cleanly.

## 7. Implementation / Source Code and Environment / Tools Used

| Environment / Tool   | Details                                       |
|----------------------|-----------------------------------------------|
| Programming Language | Java (OpenJDK 21 LTS / Java SE)               |
| Programming Style    | Pure Core Java / Object-Oriented Architecture |

| Environment / Tool | Details                                                          |
|--------------------|------------------------------------------------------------------|
| Input Method       | java.util.Scanner                                                |
| Collections Used   | ArrayList, HashMap, LinkedList, Queue, Iterator, ListIterator    |
| Concurrency API    | java.lang.Thread, java.lang.Runnable, synchronized monitor locks |
| Compilation Tool   | javac -d bin src/com/hospital/*.java                             |
| Execution Tool     | java -cp bin com.hospital.Main                                   |
| External Database  | Not used (In-memory Java Collection data structures)             |
| External Libraries | Not used (100% Standard Java Runtime)                            |

## 8. Test Cases and Expected / Actual Results

| ID   | Test Description            | Test Input                            | Expected Result                       | Actual Result                 | Status |
|------|-----------------------------|---------------------------------------|---------------------------------------|-------------------------------|--------|
| TC01 | Register General Patient    | GeneralPatient('P01', ...)            | Fee evaluates to \$500.00             | Fee calculated: \$500.00      | Pass   |
| TC02 | Register Senior Patient     | SeniorPatient('P02', ..., 40% disc)   | Fee evaluates to \$300.00             | Fee calculated: \$300.00      | Pass   |
| TC03 | Register Insured Patient    | InsuredPatient('P03', ..., 70% cover) | Fee evaluates to \$150.00             | Fee calculated: \$150.00      | Pass   |
| TC04 | Search Non-existent Patient | Search ID 'INVALID_999'               | InvalidPatientIdException caught      | Exception caught with message | Pass   |
| TC05 | Book Available Slot         | Patient P001, Doctor D001, 10:00      | Appointment booked with status BOOKED | Appointment APT-1001 created  | Pass   |

| ID   | Test Description                     | Test Input                                  | Expected Result                       | Actual Result                      | Status |
|------|--------------------------------------|---------------------------------------------|---------------------------------------|------------------------------------|--------|
| TC06 | Book Duplicate Slot Conflict         | Patient P002, Doctor D001, 10:00            | DuplicateAppointmentException caught  | Exception caught; waitlist offered | Pass   |
| TC07 | Waitlist Enrollment                  | Patient P002 joins waitlist for D001        | Patient enqueued in doctor's waitlist | Added to Doctor waitlist queue     | Pass   |
| TC08 | Cancel with Waitlist Promotion       | Cancel Appointment APT-1001                 | APT-1001 CANCELLED; P002 promoted     | Promoted P002 to new APT-1006      | Pass   |
| TC09 | Dispense Available Medicine          | Dispense 10 units of MED001 (Stock: 45)     | Stock decreases to 35 units           | Stock decremented to 35 units      | Pass   |
| TC10 | Dispense Out-of-Stock Medicine       | Dispense 999 units of MED003 (Stock: 8)     | OutOfStockException caught            | Exception caught; stock intact     | Pass   |
| TC11 | Low Stock Alert Trigger              | Dispense 5 units of MED005 (Stock: 5 - > 0) | LowStockAlert notification generated  | Alert enqueued and dispatched      | Pass   |
| TC12 | Bidirectional ListIterator Traversal | ListIterator over appointments              | Traverses forward and backward        | Traversed both directions cleanly  | Pass   |
| TC13 | Concurrent Booking Contention        | 3 threads contend for Doctor D001 @ 10:00   | Exactly 1 thread books; 2 waitlisted  | 1 booked; 2 added to waitlist      | Pass   |
| TC14 | Thread Priority Simulation           | Threads set to priorities 1, 5, 10          | Threads execute concurrently safely   | Locks guarantee state integrity    | Pass   |

| ID   | Test Description        | Test Input            | Expected Result               | Actual Result                | Status |
|------|-------------------------|-----------------------|-------------------------------|------------------------------|--------|
| TC15 | Hospital Summary Report | Select Menu Option 13 | Comprehensive metrics printed | All summary tables displayed | Pass   |

## 9. Execution Screenshots / Outputs

The application is console based. The following representative outputs describe the execution results captured from the environment:

### 9.1 Main Menu Screen

```
=====
WELCOME TO ADVANCED HOSPITAL MANAGEMENT SYSTEM (JAVA OOP & THREADS)
=====

=====
HOSPITAL MANAGEMENT SYSTEM
=====

1. Register Patient
2. Register Doctor
3. Add Medicine
4. Book Appointment
5. Cancel Appointment
6. Search Patient
7. Update Patient
8. View Doctor Appointments
9. View Waitlist
10. View Inventory
11. Dispatch Notifications
12. Run Concurrent Booking Test
13. Generate Report
14. Exit
=====
Enter your choice (1-14):
```

### 9.2 Polymorphic Patient Registration & Consultation Fee

```
[SUCCESS] Patient registered successfully!

----- PATIENT RECORD -----
ID      Name      Age  Gender  Phone      Category      Fee ($)
-----
P104    VAMSI          65   Male    9999999999  Senior Citizen $300.00

Medical History : Cancer
Polymorphic Fee : $300.00 (Senior Citizen)
-----

All Registered Patients Table:
=====
PAT ID | NAME           | AGE | GENDER | PHONE      | TYPE           | BLOOD | FEE ($) | MEDICAL HISTORY
=====
P005   | Robert Taylor  | 73  | Male   | 9876543214 | Senior Citizen | O-     | $300.00 | Osteoarthritis
P104   | VAMSI         | 65  | Male   | 9999999999 | Senior Citizen | O+     | $300.00 | Cancer
P004   | Sophia Brown  | 28  | Female | 9876543213 | General        | AB+    | $500.00 | Migraine
P001   | John Smith    | 34  | Male   | 9876543210 | General        | O+     | $500.00 | Routine Checkup
P003   | David Miller  | 42  | Male   | 9876543212 | Insured        | B+     | $125.00 | Allergic Rhinitis
P002   | Eleanor Vance | 68  | Female | 9876543211 | Senior Citizen | A+     | $300.00 | Hypertension & Diabetes
=====
```

9.3 Custom Exception Handling (Invalid ID & Out of Stock)

```
--- 10. PHARMACY INVENTORY & DISPENSING ---

-----
MED ID | NAME           | CATEGORY           | PRICE ($) | STOCK | DOSAGE RULE
-----
MED001 | Atorvastatin 20mg | Prescription (Rx) | $14.50    | 45    | PRESCRIPTION ONLY: Dosage strictly as prescribed.
MED003 | Metformin 500mg  | Prescription (Rx) | $6.75     | 8     | (LOW STOCK!) | PRESCRIPTION ONLY: Dosage strictly as prescribed.
MED002 | Amoxicillin 500mg | Prescription (Rx) | $8.25     | 60    | PRESCRIPTION ONLY: Dosage strictly as prescribed.
MED005 | Cetirizine 10mg   | Over-The-Counter (OTC) | $4.20     | 5     | (LOW STOCK!) | OTC USAGE: Take 1 tablet as directed.
MED004 | Paracetamol 650mg | Over-The-Counter (OTC) | $3.50     | 150   | OTC USAGE: Take 1 tablet every 4-6 hours.
MED006 | Ibuprofen 400mg   | Over-The-Counter (OTC) | $5.00     | 80    | OTC USAGE: Take with food for pain relief.
-----

Options:
1. Dispense Medicine to Patient (Demonstrates OutOfStockException & Stock Synchronization)
2. Restock / Update Medicine Stock
3. Return to Main Menu
Choice (1-3): 1
Enter Medicine ID to dispense: MED003
Enter Quantity to dispense: 999
Enter Patient ID receiving medication: XYZ999

[CUSTOM EXCEPTION CAUGHT] InvalidPatientIdException: Patient with ID 'XYZ999' does not exist. [Invalid ID: XYZ999]
```

9.4 Concurrent Booking Contention & Thread Priority Execution

```
Thread 1: Name=Worker-Thread-MIN-P001, Priority=1 (MIN_PRIORITY)
Thread 2: Name=Worker-Thread-NORM-P002, Priority=5 (NORM_PRIORITY)
Thread 3: Name=Worker-Thread-MAX-P003, Priority=10 (MAX_PRIORITY)

Starting concurrent booking threads simultaneously...

[Worker-Thread-NORM-P002] [Priority: 5] [Booking-SeniorPatient] Attempting to book slot 10:00 on 2026-10-01 for Patient P002 with Doctor D001...
[Worker-Thread-MIN-P001] [Priority: 1] [Booking-GeneralPatient] Attempting to book slot 10:00 on 2026-10-01 for Patient P001 with Doctor D001...
[Worker-Thread-MAX-P003] [Priority: 10] [Booking-InsuredPatient] Attempting to book slot 10:00 on 2026-10-01 for Patient P003 with Doctor D001...
>>> SUCCESS [Worker-Thread-MAX-P003]: Booked Appointment APT-1005 for David Miller at 10:00 with Dr. Gregory House (Fee: $125.00)
--- CONFLICT [Worker-Thread-NORM-P002]: Slot 10:00 on 2026-10-01 is ALREADY BOOKED! (Doctor slot is already booked for another patient. (Doctor: D001))
Waitlist...
>>> WAITLISTED [Worker-Thread-NORM-P002]: Patient P002 successfully added to Dr. D001's waitlist queue.
--- CONFLICT [Worker-Thread-MIN-P001]: Slot 10:00 on 2026-10-01 is ALREADY BOOKED! (Doctor slot is already booked for another patient. (Doctor: D001))
Waitlist...
>>> WAITLISTED [Worker-Thread-MIN-P001]: Patient P001 successfully added to Dr. D001's waitlist queue.

All concurrent booking threads have terminated.
Doctor's updated schedule and waitlist:
Appointments for Dr. Gregory House (D001 - Diagnostic Medicine & Cardiology):
```

### 9.5 Automatic Waitlist Promotion on Appointment Cancellation

```
Enter Appointment ID to cancel (e.g. APT-1001): APT-1005
Appointment APT-1005 successfully CANCELLED.
>>> WAITLIST PROMOTION: Promoted waitlisted patient Eleanor Vance (P002) to newly opened slot (2026-10-01 10:00). New Apt ID: APT-1006
```

### 9.6 Asynchronous Notification Dispatcher

```
Enter your choice (1-14): 10
--- 10. PHARMACY INVENTORY & DISPENSING ---
MED ID | NAME | CATEGORY | PRICE ($) | STOCK | DOSAGE RULE
-----|-----|-----|-----|-----|-----
MED001 | Atorvastatin 20mg | Prescription (Rx) | $14.50 | 45 | PRESCRIPTION ONLY: Dosage strictly as directed by authorized
MED003 | Metformin 500mg | Prescription (Rx) | $6.75 | 8 (LOW STOCK!) | PRESCRIPTION ONLY: Dosage strictly as directed by author
MED002 | Amoxicillin 500mg | Prescription (Rx) | $8.25 | 60 | PRESCRIPTION ONLY: Dosage strictly as directed by authorized
MED005 | Cetirizine 10mg | Over-The-Counter (OTC) | $4.20 | 5 (LOW STOCK!) | OTC USAGE: Take 1 tablet at bedtime for allergies.
MED004 | Paracetamol 650mg | Over-The-Counter (OTC) | $3.50 | 150 | OTC USAGE: Take 1 tablet every 6h for fever/pain.
MED006 | Ibuprofen 400mg | Over-The-Counter (OTC) | $5.00 | 80 | OTC USAGE: Take with food for pain/inflammation.
```

### 9.7 Comprehensive Hospital Summary Report

Smart Hospital Management System - Applet GUI

Smart Hospital Management System (Applet GUI)

Java OOP & GUI Implementation

PatientsDoctorsAppointmentsMedical Reports & FilesPharmacy & InventoryConcurrency DemoReports & Summary

Register New Patient

Patient ID:

e.g. P103

Full Name:

e.g. John Doe

Age:

e.g. 35

Phone:

e.g. 9876543210

Category:

Senior Patient (\$300 - 40% Off)

Doctor Name:

Dr. Priya

Or Custom Doctor Name:

e.g. Dr. Sarah Jenkins

Attach File:

Select file or type path.

Choose File...

Register Patient

| ID   | Name    | Age | Phone      | Category      | Doctor Na... | Attached ...  | Fee      |
|------|---------|-----|------------|---------------|--------------|---------------|----------|
| P101 | pradeep | 56  | 7893286420 | Senior (40... | bulu         | C:\Users\j... | \$300.00 |

Open Selected Attached File

## 10. Results and Validation

The project was compiled successfully using javac and validated through interactive execution and automated test suites. All functional and non-functional requirements were verified.

| Requirement / Concept            | Implementation Evidence                                                                             | Validation Status |
|----------------------------------|-----------------------------------------------------------------------------------------------------|-------------------|
| OOP Encapsulation & Abstraction  | Person, Patient, Medicine, Notification abstract hierarchies with private fields                    | Verified          |
| Runtime Polymorphism             | Differentiated fee calculations, prescription/OTC rules, and polymorphic notifications              | Verified          |
| Collections & Generics           | HashMap, ArrayList, LinkedList, and Queue utilized with type safety                                 | Verified          |
| Iterator & ListIterator          | Explicit traversal, safe removal via Iterator, bidirectional traversal and updates via ListIterator | Verified          |
| Custom Exception Handling        | Checked InvalidPatientIdException, DuplicateAppointmentException, and OutOfStockException           | Verified          |
| Multithreading & Synchronization | BookingThread and NotificationDispatchThread execute concurrently with lock safety                  | Verified          |
| Thread Priority Evaluation       | Thread priorities 1, 5, 10 evaluated and documented as OS scheduling hints                          | Verified          |
| Automated Test Execution         | QuickTest.java and Main --test pass 100% of test cases without errors                               | Verified          |

## 11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

1. In-Memory Collections vs. Database Persistence: In-memory Java Collections (HashMap, ArrayList, LinkedList) were selected because the assignment focuses on fundamental Object-Oriented design, collections manipulation, and thread memory synchronization. This avoids external JDBC and SQL setup overheads while highlighting core Java engineering.

2. Synchronization Granularity: Fine-grained synchronization on specific domain lock objects (appointmentLock, notificationLock) and synchronized methods in Doctor and Inventory was

chosen over coarse-grained global locking. This maximizes concurrency while ensuring 100% safety against race conditions during simultaneous bookings and inventory deductions.

3. Thread Priorities vs. Synchronization Locks: Testing verified that setting `Thread.MAX_PRIORITY` does not guarantee that a thread will finish or acquire a lock before a `MIN_PRIORITY` thread. Thread priority is merely a non-binding hint to the OS scheduler. System correctness must always be enforced using synchronized monitors rather than thread priorities.

4. Queue-Based Waitlist and Notifications: `LinkedList` was selected for FIFO waitlist queuing and notification processing, providing  $O(1)$  insertion and removal for fair patient promotions.

## 12. Broader Considerations / SDG Relevance

- SDG 3 (Good Health and Well-being): Optimizing appointment scheduling, eliminating double bookings, preventing medicine stockouts, and automating waitlists enhances healthcare delivery efficiency and patient safety.
- SDG 9 (Industry, Innovation, and Infrastructure): Implements scalable, thread-safe software architectural patterns suitable for mission-critical medical infrastructure.
- SDG 10 (Reduced Inequalities): Enforces fair, automated FIFO waitlist promotion without favoritism and provides institutional fee waivers for senior citizens and low-income insured patients.

## 13. Conclusion, Limitations and Possible Improvements

### 13.1 Conclusion

The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System demonstrates how Core Java can be effectively applied to model complex healthcare operations. Through OOP inheritance, polymorphism, collections, custom exceptions, and multithreading synchronization, the application delivers a robust, crash-resilient solution.

### 13.2 Limitations

- Data is volatile and resets upon application termination.
- User interaction is console-based rather than through a graphical or web interface.
- Alert notifications are simulated in the console rather than delivered via SMS or email protocols.

### 13.3 Possible Improvements

- Integrate persistent RDBMS storage using JDBC / JPA Hibernate with MySQL or PostgreSQL.
- Develop a modern Graphical User Interface (JavaFX / Swing) or Spring Boot REST API.
- Integrate real-time notification services via Twilio SMS API and JavaMail SMTP.

- Implement role-based access control (Admin, Physician, Pharmacist, Patient portals).

#### 14. Individual Contribution of Group Members

| Member Name / Roll No.       | Contribution Area                               | Evidence / Responsibility                                                                                                             |
|------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| P. VENKATA SAI PRADEEP REDDY | Core Architecture, Person & Patient Hierarchies | Person.java, Patient.java, GeneralPatient.java, SeniorPatient.java, InsuredPatient.java, Encapsulation & Polymorphic fee calculation. |
| B. HEMANTH KUMAR             | Doctor Schedules, Appointments & Waitlist Queue | Doctor.java, Appointment.java, HospitalManagementSystem.java, Iterator/ListIterator traversal, and automatic waitlist promotion.      |
| A. VAMSI                     | Pharmacy Inventory, Multithreading & Exceptions | Medicine.java, Inventory.java, BookingThread.java, NotificationDispatchThread.java, Custom Exceptions, and Synchronization locks.     |

## 14.1 ONE PAGE WRITE UP – A. VAMSI

My contribution to the Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System focused on pharmacy inventory management, multithreading, synchronization, and custom exception handling. I worked on the modules that make medicine operations safe and consistent while also supporting concurrent appointment processing and background notifications.

For the pharmacy module, I contributed to `Medicine.java` and `Inventory.java`. The medicine hierarchy supports Prescription and Over-The-Counter medicines through inheritance and polymorphism. Inventory operations support adding and displaying medicines, dispensing stock, restocking, and detecting low-stock conditions. Stock deduction is handled atomically so invalid or simultaneous operations do not produce inconsistent quantities. `OutOfStockException` provides clear feedback when the requested quantity is unavailable, while low-stock conditions can generate a `LowStockAlert` notification.

I also implemented the concurrency-related components, including `BookingThread.java` and `NotificationDispatchThread.java`. `BookingThread` simulates multiple users attempting to book the same doctor slot, while `NotificationDispatchThread` processes appointment reminders and pharmacy alerts asynchronously. Thread priorities of 1, 5, and 10 were used to demonstrate Java thread scheduling. This work showed that thread priority is only a scheduling hint and that application correctness must be guaranteed through synchronization.

To prevent race conditions, synchronization locks were applied to shared appointment and notification state. The `appointmentLock` protects simultaneous booking operations, and the `notificationLock` protects notification-queue processing. Custom checked exceptions such as `InvalidPatientIdException`, `DuplicateAppointmentException`, and `OutOfStockException` allow domain errors to be handled gracefully without terminating the application. These mechanisms improve the reliability and crash-resilience of the console-based system.

I contributed to testing and validation of medicine dispensing, out-of-stock handling, low-stock alerts, concurrent booking contention, thread-priority execution, and synchronization integrity. The documented test cases passed successfully. Overall, my work applied Java collections, inheritance, polymorphism, exception handling, `Runnable`, multithreading, thread priorities, and synchronization to a practical hospital-management problem. This contribution helped ensure safe pharmacy operations, reliable concurrent processing, and consistent asynchronous notification behavior.

## 15. References

- 1. Oracle Corporation. Java SE Documentation and java.util Collections API Reference.
- 2. Oracle Corporation. Java Concurrency and Synchronization Specifications (The Java Tutorials).
- 3. United Nations. Sustainable Development Goals (SDG 3, SDG 9, SDG 10).
- 4. Project Source Code: Hospital Management System (com.hospital).

## F. Common Assessment Rubric

| Assessment Criterion                                 | Marks | Maximum Marks |
|------------------------------------------------------|-------|---------------|
| Problem understanding & formulation                  | 10    | 10            |
| Application of course/domain knowledge               | 20    | 20            |
| Solution methodology / design / implementation       | 20    | 20            |
| Use of appropriate modern tools / techniques         | 10    | 10            |
| Results, testing & validation                        | 15    | 15            |
| Analysis, trade-offs & justification                 | 15    | 15            |
| Broader considerations / professional responsibility | 5     | 5             |
| Technical documentation & reflection                 | 5     | 5             |
| TOTAL                                                | 100   | 100           |

## G. CO-PO-Assessment Mapping

| Assessment Component               | CO            | PO(s)    | Bloom's Level | Marks |
|------------------------------------|---------------|----------|---------------|-------|
| Problem formulation                | CO1           | PO1, PO2 | L3/L4         | 10    |
| Application of knowledge           | CO1, CO2      | PO1, PO3 | L3/L4         | 20    |
| Solution / Design / Implementation | CO1, CO2, CO4 | PO3, PO5 | L4/L5/L6      | 20    |

| Assessment Component       | CO       | PO(s)      | Bloom's Level | Marks |
|----------------------------|----------|------------|---------------|-------|
| Modern tool usage          | CO2, CO4 | PO5        | L3/L4         | 10    |
| Validation                 | CO3, CO4 | PO4        | L4/L5         | 15    |
| Analysis & justification   | CO2, CO4 | PO2, PO4   | L4/L5         | 15    |
| Broader considerations     | CO1      | PO6, PO7   | L4/L5         | 5     |
| Documentation & reflection | CO1, CO2 | PO10, PO12 | L3/L4         | 5     |

#### H. Faculty Evaluation & Continuous Improvement

| Performance Level | Criterion   |
|-------------------|-------------|
| Level 3           | $\geq 70\%$ |
| Level 2           | 60 - 69%    |
| Level 1           | 50 - 59%    |
| Level 0           | $< 50\%$    |

Target CO Attainment: Level 3 | Actual CO Attainment: Level 3 (100% Criteria Satisfied)